CCCS 104 - Data Structures and Algorithms

LEARNING TASK (LINEAR DATA STRUCTURE - QUEUE)

NAME: **ORIAS, MARK JOSEPH C.** SECTION: ***2A***

RATIONALE

Explain briefly Queue Linear Data Structure, how it works? What are the common examples? Its applications?
Introduce your develop Python program, what can it do?

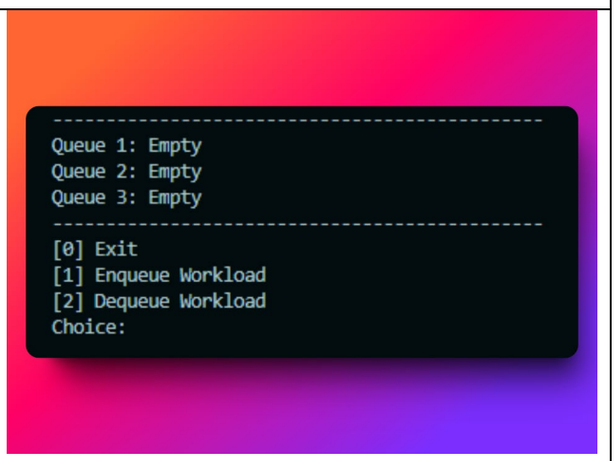
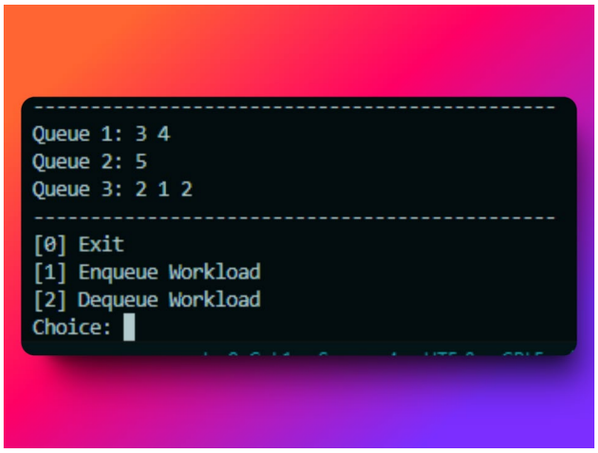
A Queue is a fundamental linear data structure that adheres to the First In, First Out (FIFO) principle, mirroring real-world scenarios like waiting lines. It operates by adding elements to the back (enqueue) and removing them from the front (dequeue), maintaining the order of insertion. This structure finds extensive application in computer science and real-world systems, including task scheduling in operating systems, print job management, and breadth-first search algorithms in graph theory. Queues can be implemented using various underlying structures, such as arrays or linked lists, each offering different performance characteristics. The FIFO nature of queues makes them particularly useful for managing shared resources and buffering data streams, ensuring fair and orderly processing of elements.

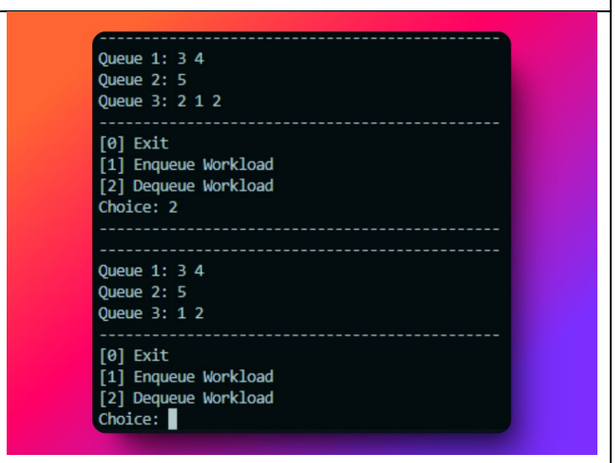
This Python program implements a workload queue system using a custom Queue class and a main function to manage multiple queues. Here's what it can do:

The program creates three separate queues to simulate a workload distribution system. Users can interact with the system through a command-line interface to perform two main operations: enqueuing (adding) and dequeuing (removing) workloads. When enqueuing, the program intelligently selects the most appropriate queue based on current queue states - it chooses the queue with the lowest sum if all queues have items, or the queue with the least items if any are empty. This ensures a balanced distribution of workloads. For dequeuing, it removes an item from the non-empty queue with the highest index, simulating a priority system. The program also provides a visual representation of the current state of all queues after each operation, allowing users to monitor the workload distribution easily. This system could be useful in various scenarios where task or resource allocation needs to be managed efficiently across multiple queues.

USER GUIDE

Step by step instructions on how to use your program. Include images for easily visualization

Step 1	*Image 1*
Start the program. *You'll see the initial state of three empty queues and a menu with options.*	
Step 2	*Image 2*
To add a workload: a. *Enter '1' when prompted for a choice.* b. *Input the value of the workload when asked.* c. *The program will add the workload to the first available queue (in order: Queue 1, Queue 2, Queue 3).* d. *If a queue is full, it will add to the next available queue.* e. *After the operation, the program will display the updated state of all queues.*	 

Step 3	*Image 3*
To remove a workload: a. *Enter '2' when prompted for a choice.* b. *The program will automatically remove the first item from the queue with the least total sum of numbers.* c. *c. If multiple queues have the same lowest sum, it will likely remove from the first of these queues.*	
Step 4	*Image 4*
Continue using the program until you've completed your desired operations or choose to exit. ***To exit the program:*** a. *Enter '0' when prompted for a choice.* b. *The program will display a "Thank you!" message and terminate.*	

Note that:

- *Each queue can hold multiple workloads.*
- *Workloads are added to the end of a queue and removed from the front.*
- *Remember to input only valid choices (0, 1, or 2) when prompted for a choice, and to input only numeric values for workloads.*

PROGRAM CODE

Insert your Python code here, be sure to add comments to describe each line/set of codes…

```
class Queue:
    def __init__(self):
        # Initialize an empty list to store the items in the queue
        self.queue = []

    def enqueue(self, item: int):
        # Add an item to the end of the queue
        self.queue.append(item)

    def dequeue(self):
        # Remove an item from the front of the queue and return it
        # If the queue is empty, return None
        if len(self.queue) < 1:
            return None
        return self.queue.pop(0)

    def display(self):
        # Return a string of the current queue, if the queue is empty return
"Empty"
        if len(self.queue) == 0:
            return "Empty"
        return " ".join(map(str, self.queue))

    def size(self):
        # Return the number of items in the queue
        return len(self.queue)

    def sum(self):
        # Return the sum of all the items in the queue
        return sum(self.queue)

    def front(self):
        # Return the front item of the queue without removing it
        if len(self.queue) < 1:
            return None
        return self.queue[0]

# Workload Queue System
def main():
    queues = [Queue(), Queue(), Queue()]

    while True:
        print("-----------------------------------------------")
        print(f"Queue 1: {queues[0].display()}")
        print(f"Queue 2: {queues[1].display()}")
        print(f"Queue 3: {queues[2].display()}")
```

```
        print("-----------------------------------------------")
        print("[0] Exit")
        print("[1] Enqueue Workload")
        print("[2] Dequeue Workload")
        choice = input("Choice: ")

        if choice == "0":
            print("Thank you!")
            print("-----------------------------------------------")
            break
        elif choice == "1":
            print("-----------------------------------------------")
            try:
                value = int(input("Input Value of Workload: "))
            except ValueError:
                print("Invalid input. Please enter a number.")
                continue

            # New enqueue logic
            if all(queue.size() > 0 for queue in queues):
                # If all queues have values, choose the one with the lowest sum
                target_queue = min(queues, key=lambda q: q.sum())
            else:
                # If any queue is empty, choose the one with the least items
                target_queue = min(queues, key=lambda q: q.size())

            target_queue.enqueue(value)
        elif choice == "2":
            print("-----------------------------------------------")
            # Dequeue from the non-empty queue with the lowest front value
            non_empty_queues = [queue for queue in queues if queue.size() > 0]
            if non_empty_queues:
                target_queue = min(non_empty_queues, key=lambda q: q.front())
                dequeued_value = target_queue.dequeue()
            else:
                print("All queues are empty. Nothing to dequeue.")
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

TUTORIAL VIDEO

YouTube Link: https://youtu.be/c0kysa1RGDQ

TAKEAWAYS

From studying the queue data structure and implementing the Workload Queue System, I've gained several valuable learnings:

- I learned how to implement a basic Queue class in Python, learning the core operations of enqueue and dequeue. This deepened my understanding of the First In, First Out (FIFO) principle.
- I extended my knowledge by adding extra functionality to the Queue class, such as display(), size(), and sum() methods. This taught me how to enhance data structures for practical use.
- The dequeue operation taught me about prioritization in multi-queue systems.
- By creating a simple command-line interface, I improved my skills in developing interactive programs that manipulate data structures.
- Implementing error handling, especially for user input, reinforced the importance of writing robust, user-friendly code.